System Design and Architecture

Huru SMS

Multichannel SMS and Web Citizen Information and Guidance System for Tanzania

This document describes how Huru SMS is designed and built: the parts of the system, how they work together, how a question travels from a citizen's phone to an answer, how data is stored, how the system is deployed and secured, and what the citizen and the operator see on screen.

Owner and developer: Huru Digital Co. Ltd., Tanzania

Work: Computer software (web application and SMS service)

Version described: 1.0

Date: 9 October 2026

Prepared for: Copyright Society of Tanzania (COSOTA)

Companion documents: *Huru SMS – Innovation Documentation* and *Huru SMS – Source Code Listing*

1. Purpose of this document

The *Innovation Documentation* explains what Huru SMS is and why it was created. This document explains **how it is built**. It is written for a reader who wants to verify that the submitted source code is a complete, working, original system. It contains:

- a system overview showing the people and outside services the system talks to (Section 2);
- the functional design: what each part of the system does (Section 3);
- the software architecture: layers, components and the source files that implement them (Section 4);
- the processing flows for an SMS question, a web question and the answer pipeline (Section 5);
- the data design: database tables and their relationships (Section 6);
- the deployment and integration design (Section 7) and the security design (Section 8);
- the technology stack (Section 9) and screenshots of the SMS interface, the web interface and the administration panel (Section 10).

File names in this document (for example `app/Services/ConversationService.php`) refer to the files reproduced in the *Source Code Listing*.

2. System overview

Huru SMS is a server application. Citizens reach it in two ways: by sending an SMS starting with the keyword **HURU** to the shortcode **15054**, or by opening the web chat in a browser. Staff of Huru Digital manage the content and settings through a protected administration panel. The application uses three outside services: the SMS gateway that carries messages to and from the mobile networks, a language model service that words the final reply, and a public directory of legal aid providers that is imported periodically.

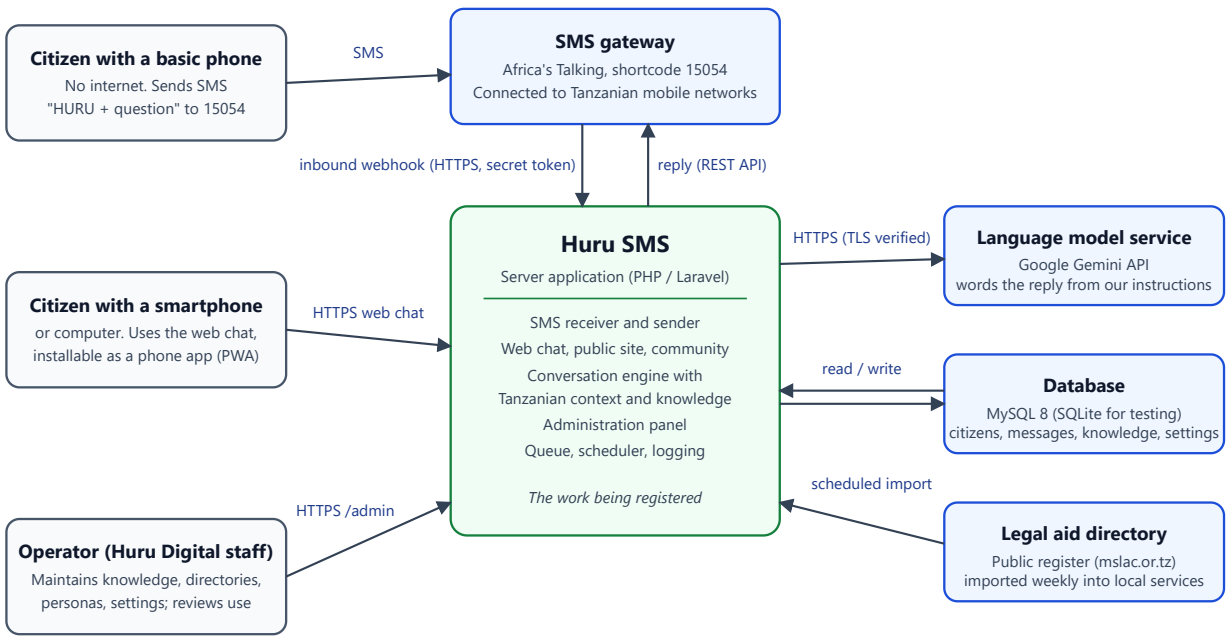


Figure 1. System context: the people and outside services connected to Huru SMS.

Actor or service	Role in the system
Citizen (SMS)	Sends questions and service commands by SMS from any phone. Identified only by phone number.
Citizen (web)	Logs into the web chat with a phone number confirmed by a one-time SMS code. Can search history, rate answers, set name, language and region, and take part in the community board.
Operator	Signs in to the administration panel with e-mail and password. Maintains knowledge entries, the national directory, local services, personas (prompt templates) and system settings; reviews conversations and engine logs; blocks abusive users.
SMS gateway	Africa's Talking delivers inbound SMS to the webhook and sends outbound replies. Any gateway with a similar HTTP interface can replace it; the integration is isolated in one class.
Language model service	Receives the system instructions and conversation turns assembled by Huru SMS and returns the worded reply. It never receives the citizen's name or number. The provider is behind an interface and can be swapped.
Database	Stores citizens, messages, engine logs, feedback, knowledge, directories, templates, settings, community content, sessions, cache and the job queue.

3. Functional design

3.1 Modules

Module	Functions	Main source files
SMS channel	Receives inbound SMS from the gateway, verifies the secret token, removes the keyword, queues the message, processes it and sends the reply. Handles delivery reports.	`routes/api.php`, `Http/Controllers/SmsController.php`, `Http/Middleware/VerifySmsWebhook.php`, `Jobs/ProcessIncomingSms.php`, `Services/SmsService.php`
Web chat	Phone-number login with one-time code, conversation screen, history search by keyword, date and topic, helpful / not helpful rating, preferences (name, language, region), offline notice, installable app.	`Http/Controllers/WebChatController.php`, `resources/views/chat.blade.php`, `public/sw.js`, `public/manifest.json`

Conversation engine	One pipeline for both channels: service commands, opt-out, bans, rate limit, topic classification, emergency detection, abuse moderation with strikes, Tanzanian context, knowledge retrieval, local resources, national contacts, history, reply generation, cleaning, storing and logging.	`Services/ConversationService.php`, `TopicClassifier.php`, `LanguageDetector.php`, `TanzaniaContext.php`, `KnowledgeRetriever.php`, `PromptEngine.php`, `AiService.php`, `ModerationService.php`, `Ai/GeminiClient.php`
Public site and community	Landing page with the SMS instructions, topics and contact form; terms and privacy pages; community threads where logged-in citizens post, reply, react and join or create groups.	`routes/web.php`, `views/welcome.blade.php`, `Http/Controllers/CommunityController.php`, `views/community/*`, `views/legal/*`
Administration panel	Dashboard (questions per day, topics, channels, tokens, errors), conversations and engine logs, knowledge base with file import, personas, national directory with verification, local services, citizens (ban, reset strikes), settings grouped by Branding, Engine, Channels and Safety.	`Http/Controllers/Admin/*`, `Http/Middleware/EnsureAdmin.php`, `views/admin/*`, `Support/Settings.php`
Background tasks	Database queue for SMS processing, a scheduler that drains the queue every minute and imports the legal aid directory weekly.	`routes/console.php`, `Console/Commands/FetchLegalAidProviders.php`
Starter data and tests	Seeders for settings, the administrator, personas, 42 knowledge entries, 33 national contacts and local services; 28 automated tests of the SMS flow, web chat, community, classifier, language and location detection and prompt assembly.	`database/seeders/*`, `tests/*`

3.2 Service commands (both channels)

Command	Effect
`MSAADA`, `HELP`, `MENU`, `?`	Short guide to the service in the citizen's language
`ACHA`, `STOP`, `SITAKI`, `ONDOA`	Opt out: no further replies by SMS until the citizen sends a start command
`ANZA`, `START`, `RUDI`	Resume the service

`LUGHA SW` / `LUGHA EN` / `LUGHA AUTO`	Fix the reply language to Kiswahili or English, or follow the language written
`JINA <name>`	Save the citizen's name for a personal reply
`MKOA <region or district>`	Save the citizen's region (recognises all 31 regions and common districts) so nearby services are prioritised

Commands are answered directly by the application and never reach the language model.

4. Software architecture

The application follows a layered architecture on the Laravel framework. Each layer only calls the layer below it. Both channels enter the same conversation engine, so the behaviour never differs between SMS and web.

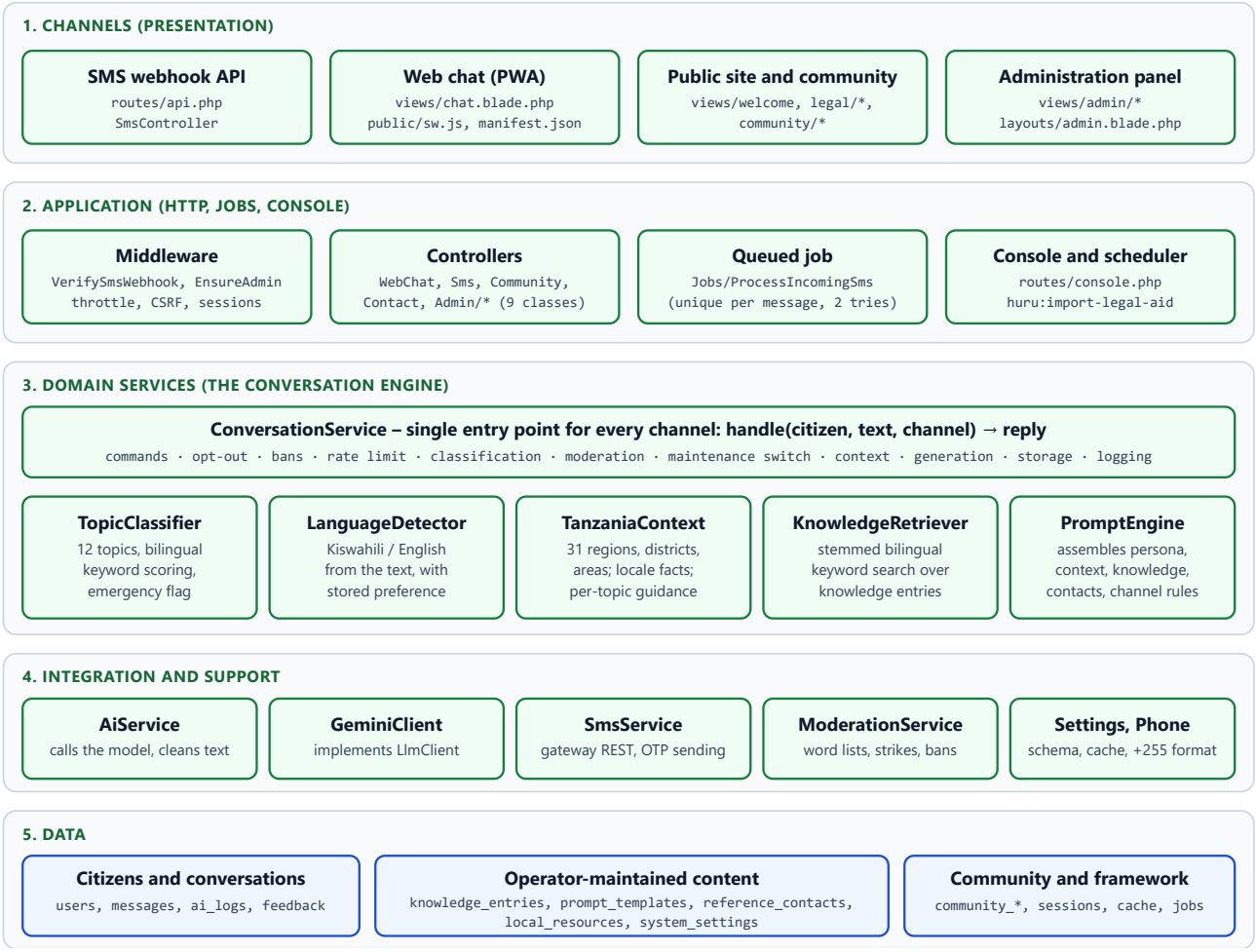


Figure 2. Layered architecture. Every box is an original class or view in the source code listing.

4.1 Design principles

- **One engine, many channels.** SMS and web both call `ConversationService::handle()`. Channel-specific limits (for example the 120-word SMS budget) are passed as context, not duplicated.
- **Context before generation.** The application decides the topic, language, location, knowledge, nearby services and official contacts *before* the language model is called. The model only words a reply inside those instructions, which keeps answers Tanzanian and prevents invented numbers or offices.

- **Replaceable providers.** The model is used through the `LlmClient` interface and the gateway through `SmsService`; either can be swapped without touching the engine.
- **Operator control without code changes.** Knowledge, directories, personas and settings live in the database and are edited in the admin panel. Settings are whitelisted in `Support/Settings.php` so only known keys with validated types can be changed.
- **Graceful degradation.** If the model, the gateway or the key is unavailable the citizen gets a polite message, the failure is logged with its cause, and commands keep working.

4.2 Component responsibilities

Component (file)	Responsibility
`Services/ConversationService.php`	Orders the pipeline in Figure 5; resolves language; applies commands, opt-out, bans, rate limit, strikes and the maintenance switch; stores messages; writes engine logs.
`Services/TopicClassifier.php`	Scores bilingual keyword lists for 12 topics (law, health, agriculture, education, government services, finance, employment, family, technology, transport, business, general) and flags emergencies such as "amezimia" or "not breathing".
`Services/TanzaniaContext.php`	Knows the 31 regions, districts and well-known areas; detects a location in free text; provides the locale block (currency, time zone, institutions, emergency numbers) and per-topic guidance in both languages.
`Services/KnowledgeRetriever.php`	Tokenises and stems Kiswahili and English, removes stop words, and ranks active knowledge entries by title, keyword and content matches for the detected topic and language.
`Services/PromptEngine.php`	Builds the system instruction from persona template, country context, topic rules, citizen profile, retrieved knowledge, nearby local resources, national contacts (emergency first) and channel constraints; builds the multi-turn history.
`Services/AiService.php`, `Services/Ai/GeminiClient.php`	Send the instruction and turns to the model with the configured model name, temperature and token limit; map safety blocks, errors and missing configuration to statuses; clean markdown and length.
`Services/SmsService.php`	Sends SMS and one-time codes through the gateway REST API (sandbox or live endpoint); reports when not configured instead of failing.
`Services/ModerationService.php`	Detects abusive wording, counts strikes per citizen, issues warnings and bans at the configured threshold.
`Http/Controllers/WebChatController.php`	Session, one-time code request and verification, message listing with filters, sending, feedback, preferences and logout; returns a fresh CSRF token after login.
`Http/Controllers/SmsController.php`, `Jobs/ProcessIncomingSms.php`	Accept the gateway callback quickly, enforce the keyword, queue the processing, then run the engine and send the reply in the background.

`Http/Controllers/Admin/*`	Dashboard statistics, conversation review, CRUD screens for knowledge (with CSV/JSON import), personas, contacts (with verification), local resources, citizens and settings.
`Support/Settings.php`, `Support/Phone.php`	Typed, validated, cached settings schema in four groups; phone normalisation to +255 format and masking for display.

5. Processing flows

5.1 An SMS question

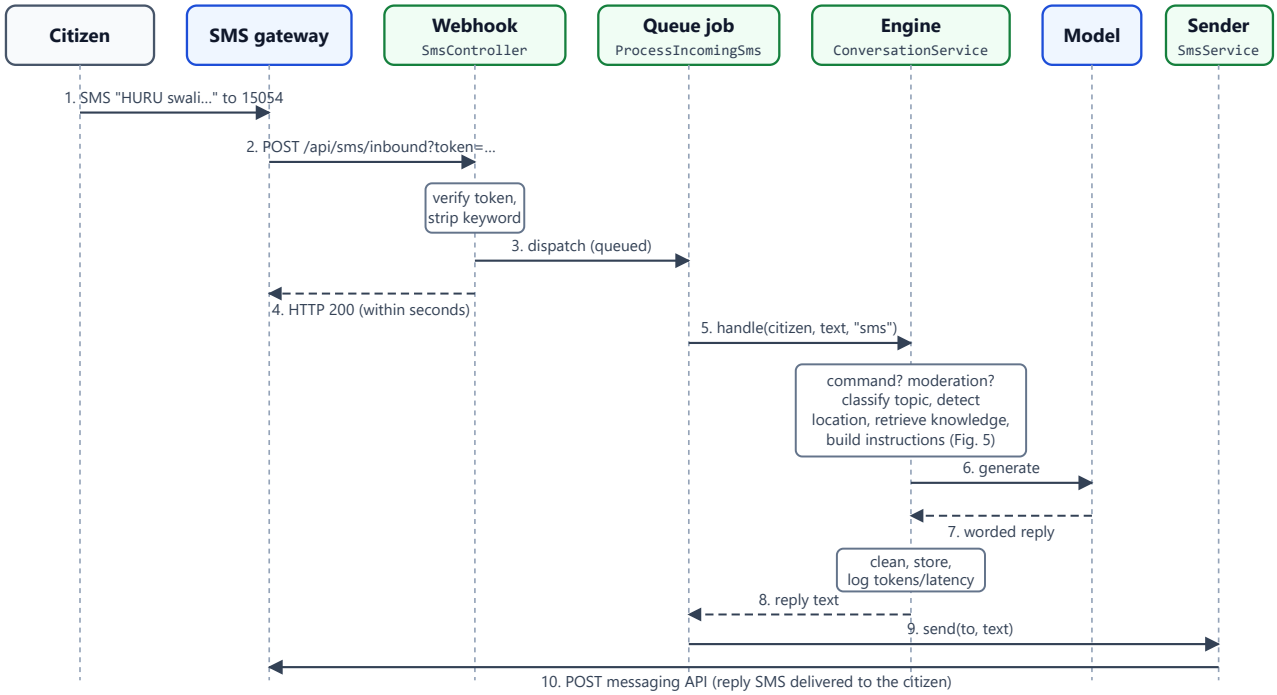


Figure 3. Sequence of an SMS question. The webhook answers the gateway immediately; the slow work happens in the queue.

5.2 A web chat question

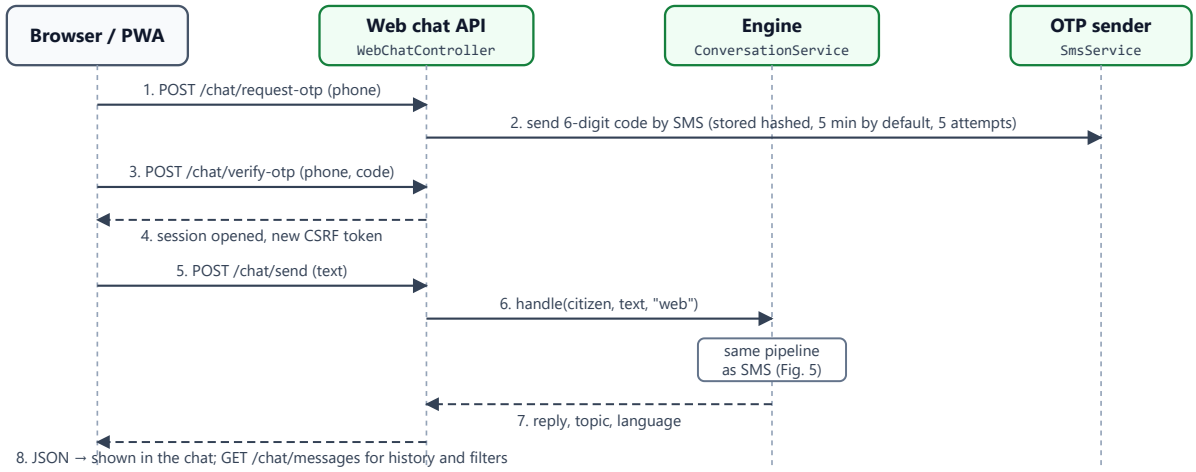


Figure 4. Sequence of a web chat login and question. The web channel shares the engine with SMS.

5.3 The answer pipeline

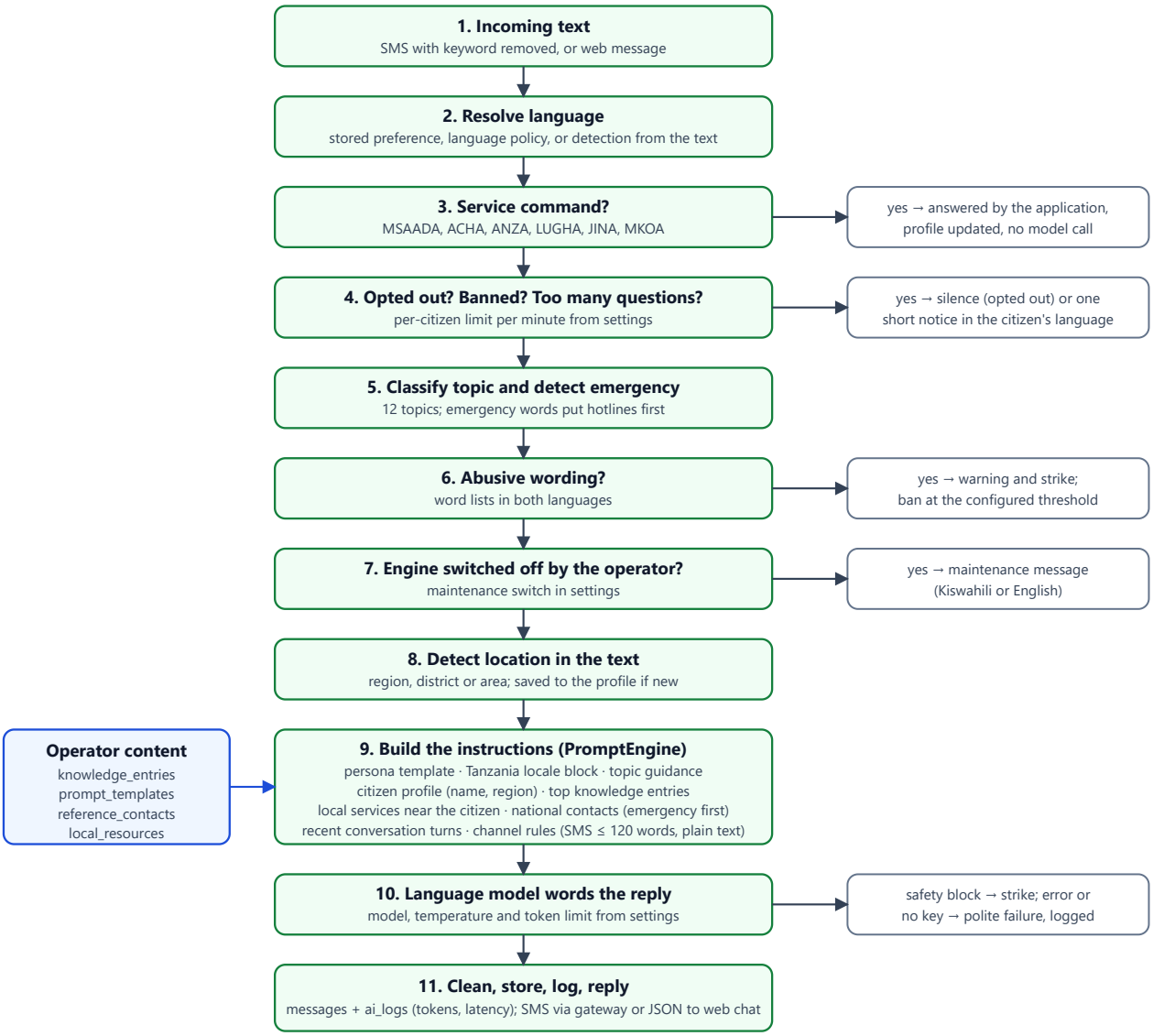


Figure 5. The answer pipeline inside the conversation engine. Steps 1 to 9 and 11 are entirely original application logic; step 10 is the only call to an outside model.

6. Data design

The database has 14 application tables plus the framework tables for sessions, cache and the job queue. Figure 6 shows the application tables and their relationships; the tables that follow describe the important columns.

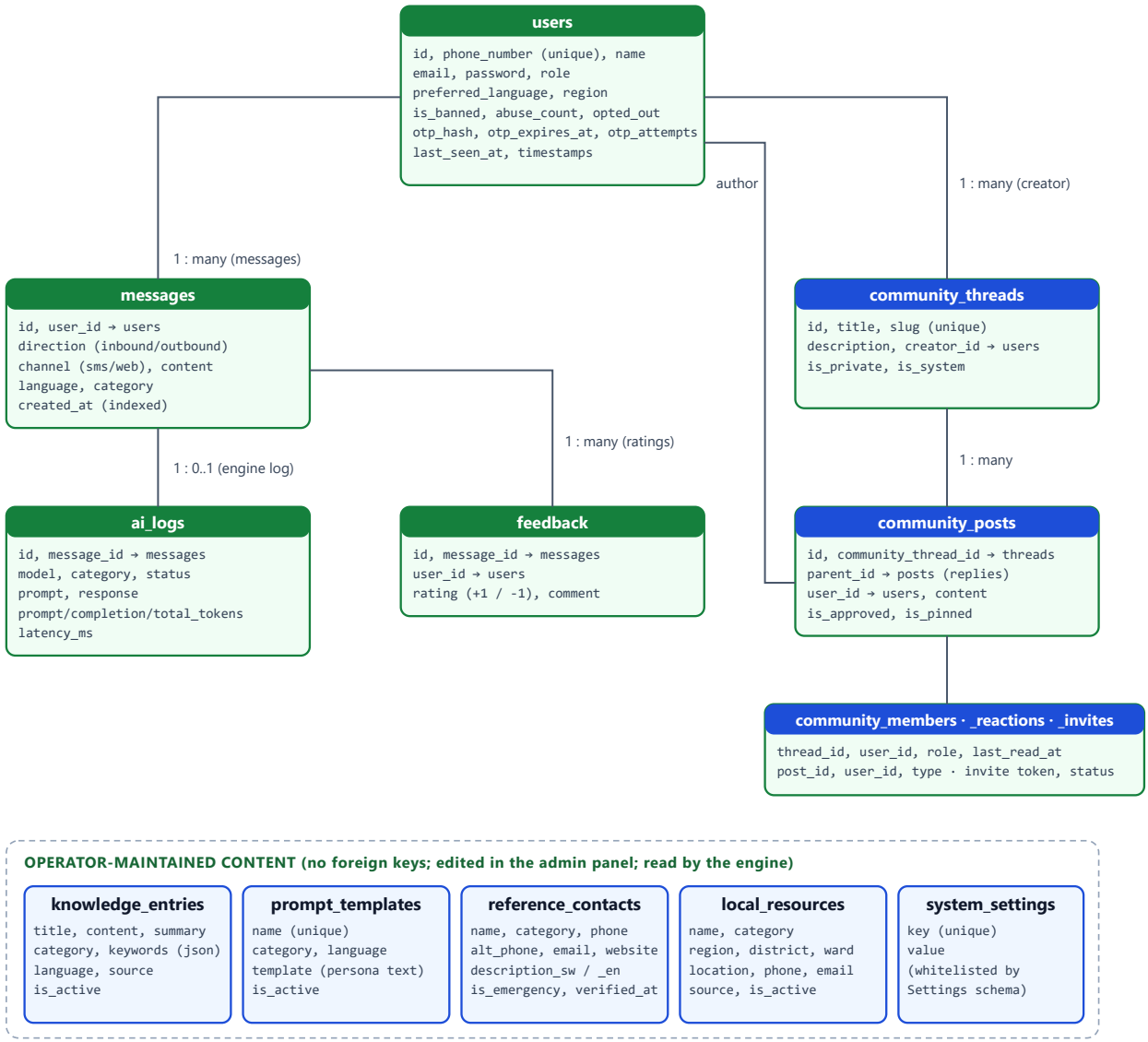


Figure 6. Entity-relationship diagram of the application tables (migrations in `database/migrations/`).

Table	Purpose and notable design choices
`users`	One row per citizen or operator. Citizens are identified by phone number in +255 format; operators by e-mail and password with `role = admin`. Holds the hashed one-time code, strike count, ban and opt-out flags, and preferences.
`messages`	Every inbound and outbound text on both channels, with language and topic. Indexed by citizen and time, and by channel and time, for history search and dashboard statistics.

`ai_logs`	One row per engine call attached to the outbound message: the full instructions sent, the raw response, status, token counts and latency. Lets the operator audit why an answer was given.
`feedback`	Helpful / not helpful rating per answer per citizen (unique pair).
`knowledge_entries`	Verified facts the engine quotes; bilingual with keywords for retrieval; can be switched off without deleting.
`prompt_templates`	Personas: the tone and rules per topic and language, editable by the operator.
`reference_contacts`	National institutions and hotlines with a verification date and an emergency flag, so emergency numbers are listed first.
`local_resources`	Services by region, district and ward (186 legal aid providers imported, plus operator entries). Indexed by region and district for nearby lookups.
`system_settings`	Key-value store; only keys defined in the settings schema are accepted and values are validated and cached.
`community_*`	Threads, membership, posts with replies, reactions and invitations for the community board.
`sessions`, `cache`, `jobs`	Laravel framework tables: web sessions, cached settings and rate limits, and the queue of SMS messages waiting to be processed.

7. Deployment and integration design

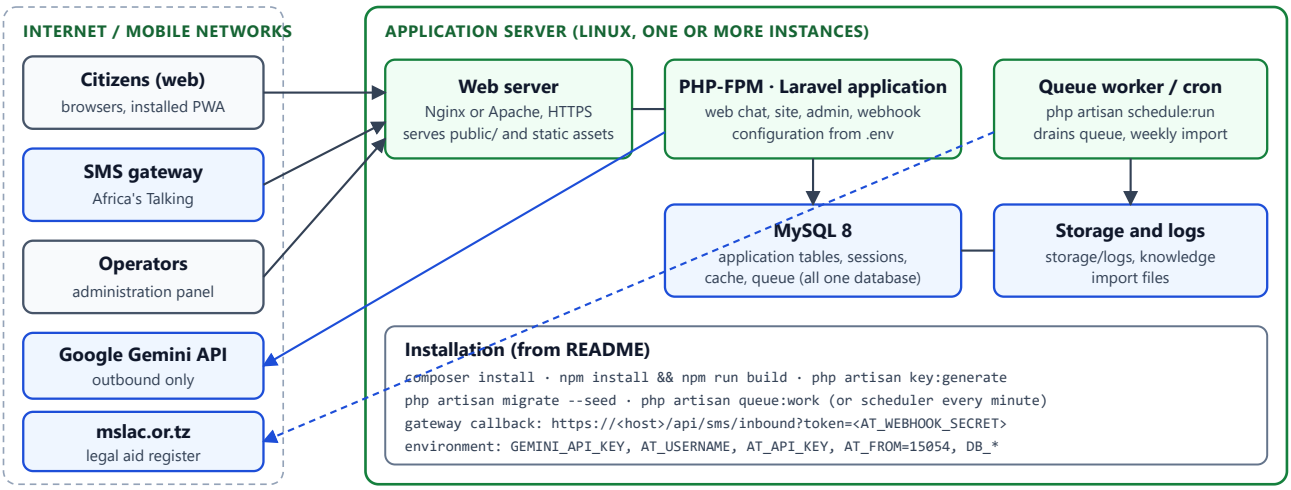


Figure 7. Deployment view. Outbound calls (blue) use verified TLS; the gateway callback is protected by a secret token.

7.1 Integration details

Integration	How it is implemented
Inbound SMS	The gateway posts `from`, `to`, `text`, `id` to `/api/sms/inbound`. The `VerifySmsWebhook` middleware compares the `token` query parameter or `X-Webhook-Token` header with the configured secret. With `AT_REQUIRE_KEYWORD` on, messages without the keyword are ignored (shared shortcodes). The keyword is removed and the message queued.
Outbound SMS	`SmsService` posts to the gateway messaging endpoint (sandbox or live chosen by the username) with the shortcode as sender. Long replies are kept within the SMS word budget by the engine.
Language model	`GeminiClient` calls `generateContent` with a system instruction and the conversation turns; model name, temperature and maximum tokens come from settings. Responses are classified as ok, safety-blocked, error or unconfigured.
TLS	All outbound HTTPS calls verify certificates using a bundled CA store, so the application behaves the same on Windows and Linux hosts.
Legal aid register	The console command `huru:import-legal-aid` downloads the public provider list, maps names, regions, districts and addresses to `local_resources`, and runs weekly from the scheduler.
Offline web use	A service worker caches the application shell and shows an offline page with the SMS instructions; chat, admin and API responses are never cached.

8. Security and privacy design

Concern	Design
Citizen login (web)	Phone number plus one-time SMS code; the code is stored hashed, expires after a set number of minutes, allows five attempts, and requests are rate-limited per number. The session is regenerated and the CSRF token rotated after login.
Operator login	E-mail and hashed password; login attempts throttled; every admin route requires an authenticated user with the admin role (`EnsureAdmin`).
Gateway callback	Secret token required on every inbound SMS and delivery report; requests without it are rejected before any processing.
Request protection	CSRF tokens on all state-changing web requests; per-route throttling on OTP, sending, posting and login; input validation on every controller.
Abuse and safety	Word-list moderation with warnings, strikes and bans; model safety blocks count as strikes; emergencies surface official hotlines first; per-citizen rate limit.
Data minimisation	Only the phone number and the conversation are stored. The name and number are never sent to the model. Phone numbers are masked in the admin panel and in logs.
Secrets	API keys, the webhook secret and database credentials live only in the `.env` file, which is excluded from the deposited package and from version control.
Settings integrity	Only keys declared in the settings schema can be written; each value is validated against its declared type and options.
Transport	HTTPS for citizens and operators; verified TLS for all outbound calls.

9. Technology stack

Layer	Technology
Language and framework	PHP 8.2+ with Laravel 12 (routing, Eloquent ORM, queue, scheduler, sessions, validation, testing)
Database	MySQL 8 in production; SQLite for local testing; migrations define every table
Front end	Blade templates, hand-written CSS and JavaScript, no front-end framework; installable PWA with service worker
SMS	Africa's Talking HTTP API (shortcode 15054, keyword HURU), called directly with Laravel's HTTP client
Language model	Google Gemini `generateContent` API behind the `LlmClient` interface
Build and tests	Composer, npm with Vite and Tailwind for asset building, PHPUnit (28 tests, 135 assertions)
Size of the original code	112 files, 7,991 lines (see the Source Code Listing)

10. Screenshots of the user interfaces

All screenshots were captured on 9 October 2026 from a running installation of version 1.0 with sample data. The phone number shown belongs to a sample citizen "Asha" in Shinyanga and is fictitious. The SMS views show the exact messages exchanged with the service as they appear in a phone's messaging application. Answers in the sample conversation illustrate the format produced by the engine.

10.1 SMS interface (any phone, no internet)

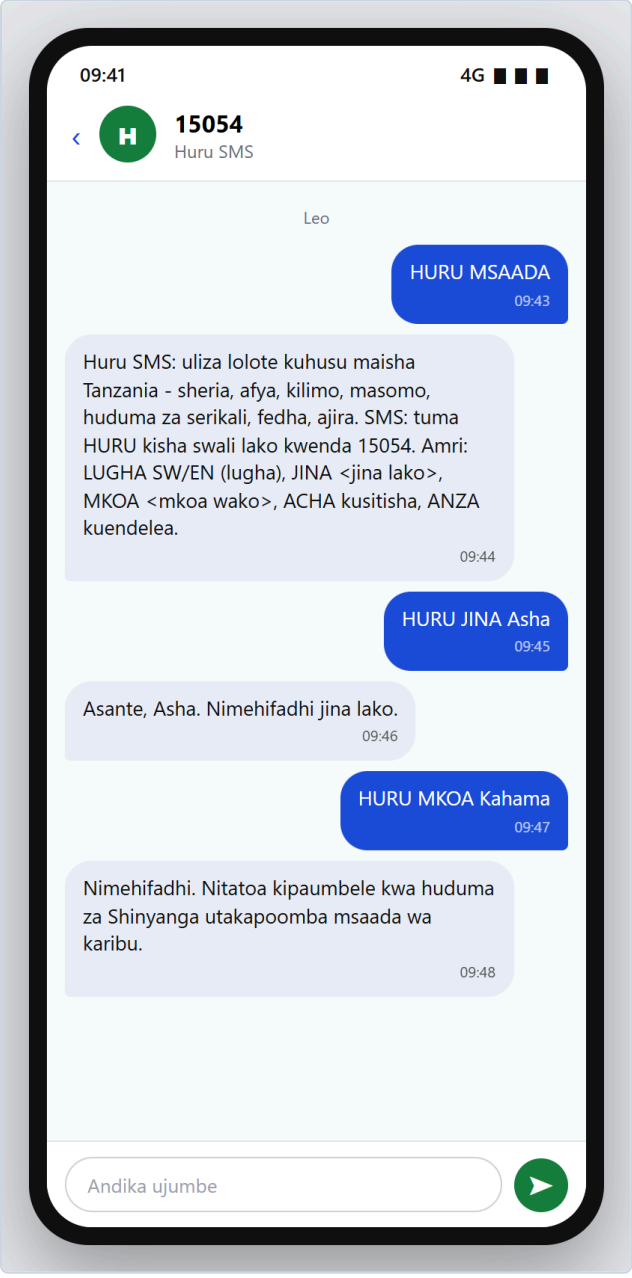


Figure 8. Service commands by SMS: MSAADA (help), JINA (name), MKOA (region). All replies are generated by the application.

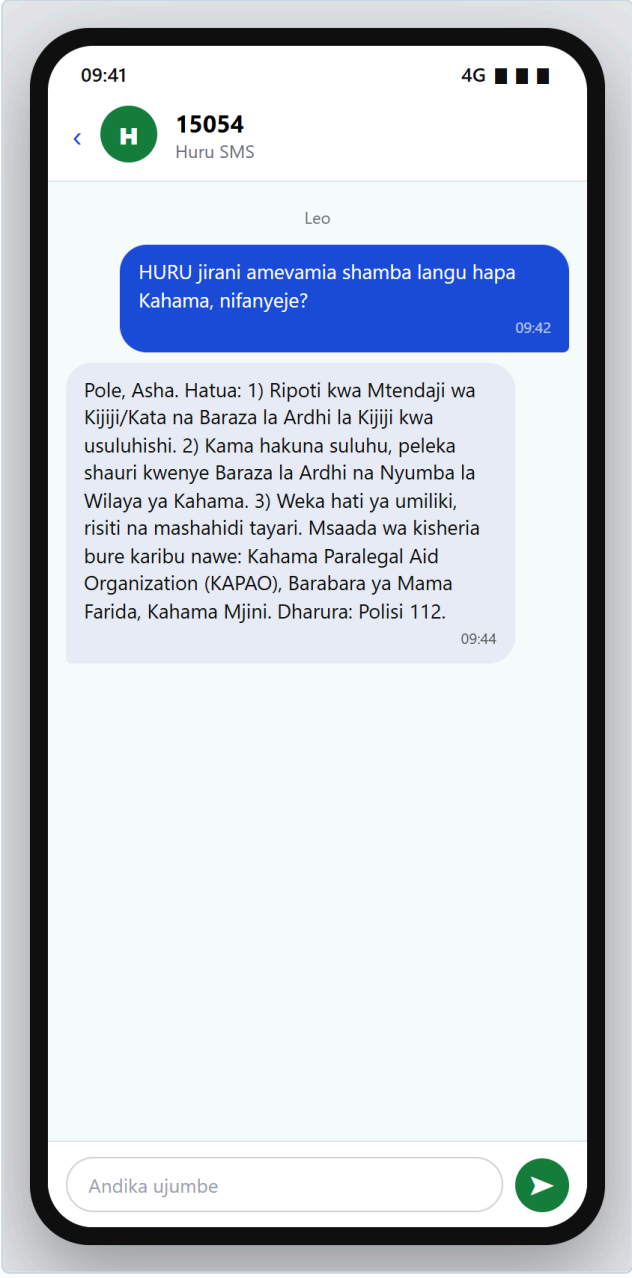


Figure 9. A question by SMS with the keyword HURU to 15054 and the reply, which names a legal aid provider in Kahama and the emergency number.

10.2 Web chat interface

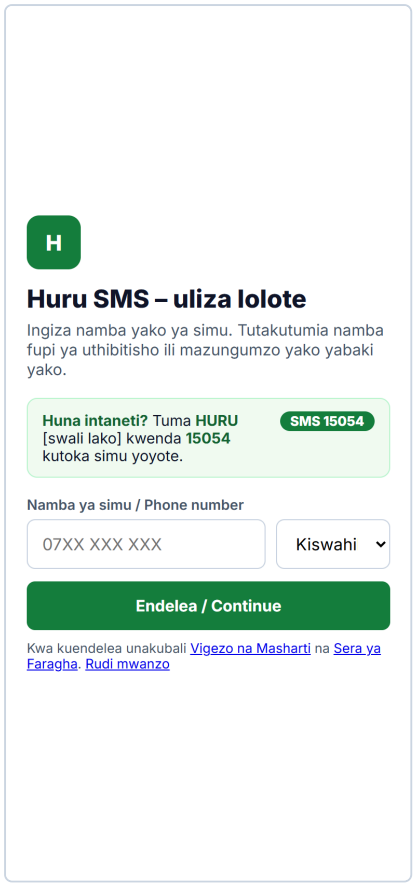


Figure 10. Web chat login on a phone: enter the phone number and choose the language.

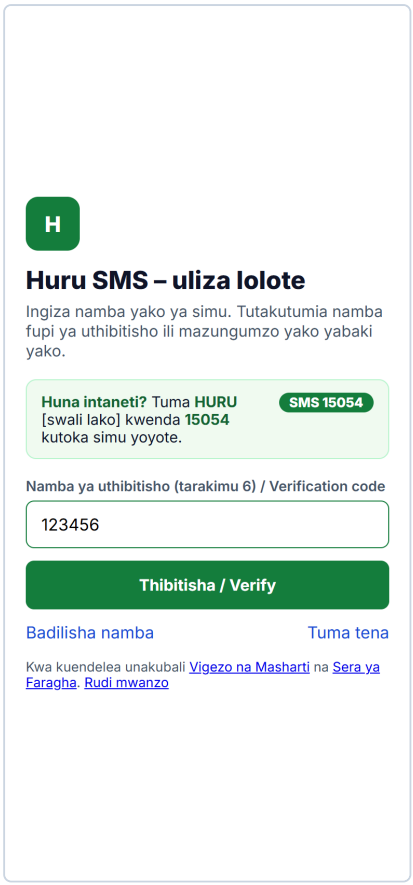


Figure 11. Entering the one-time code received by SMS.

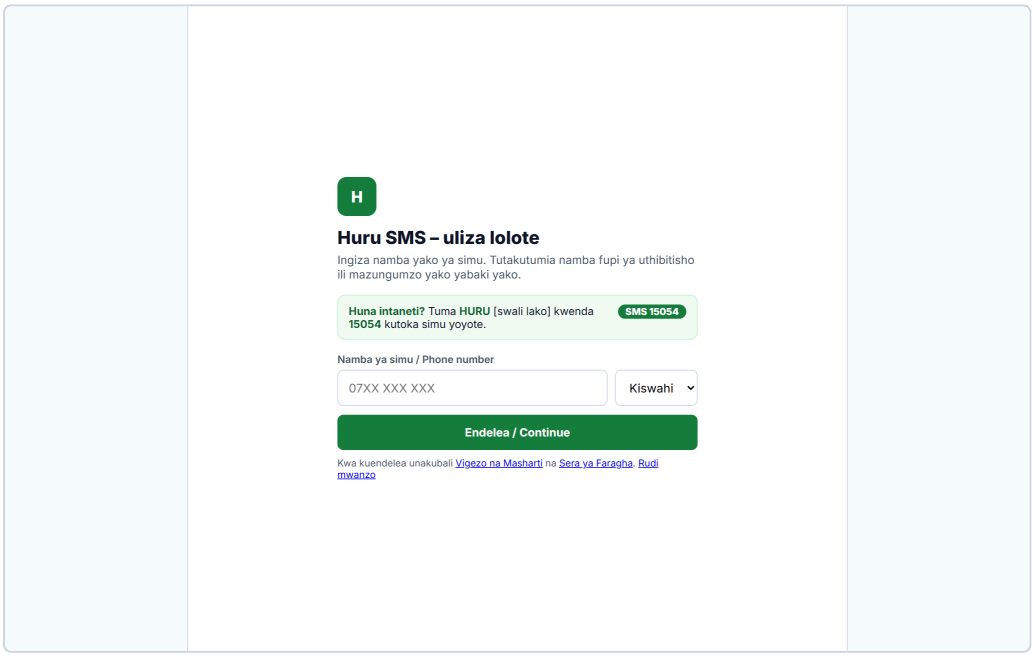


Figure 12. The same login screen on a computer, showing the SMS alternative (HURU to 15054).

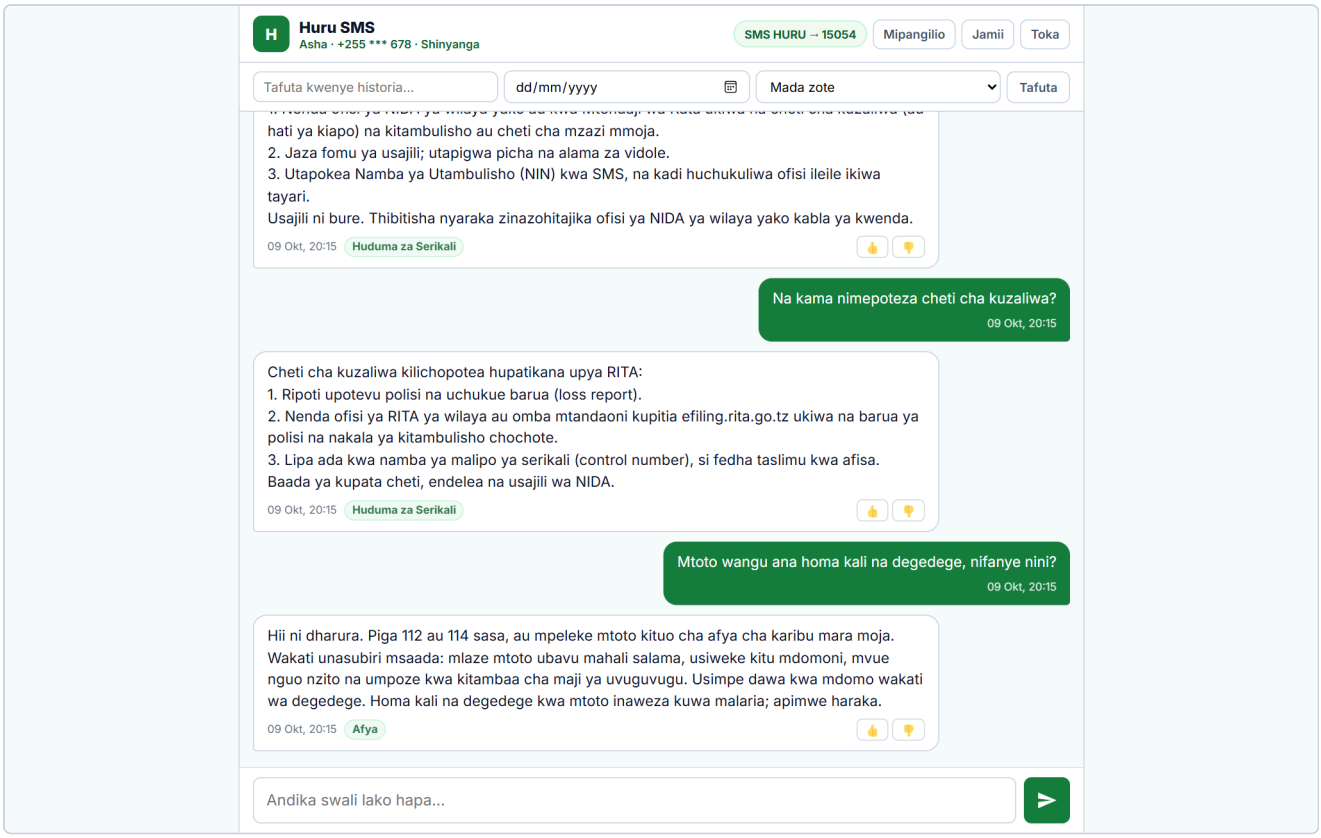


Figure 13. A conversation on a computer: topic labels on each answer, helpful / not helpful buttons, and history search by keyword, date and topic.

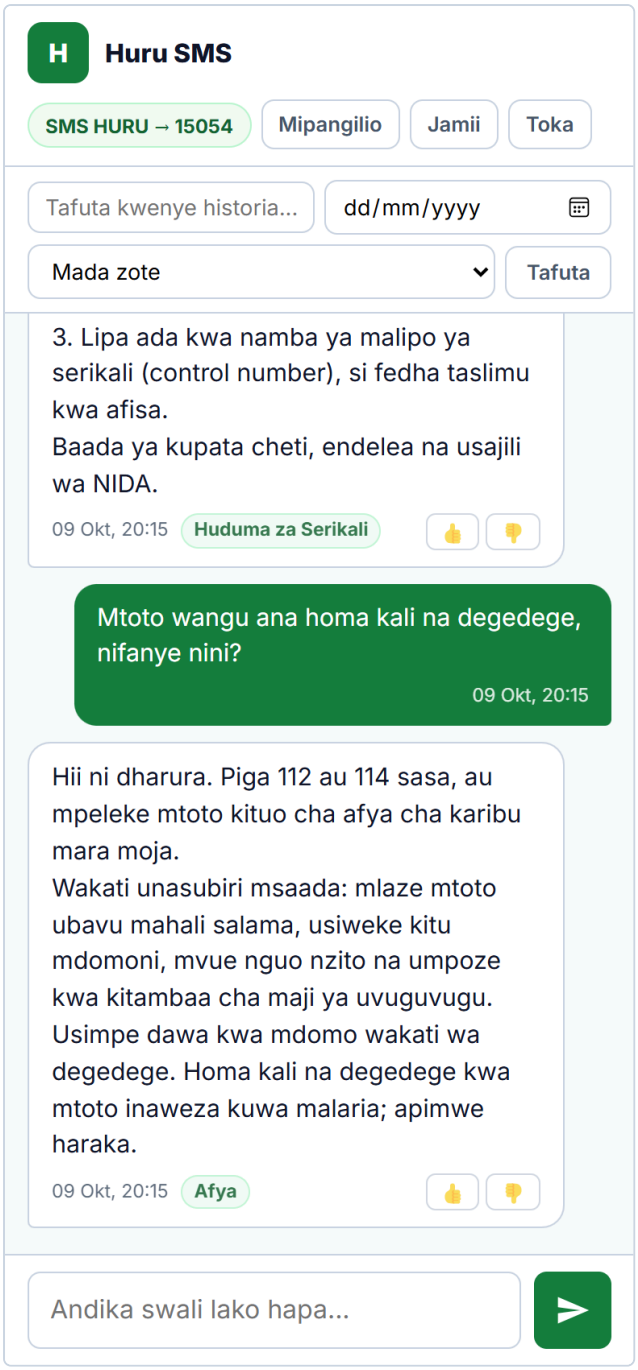


Figure 14. The same conversation on a phone, including an emergency answer that gives the hotline first.

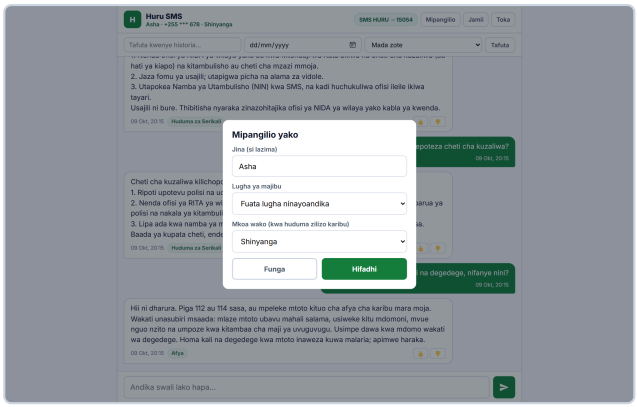


Figure 15. Preferences: name, reply language and region for nearby services.

10.3 Public website

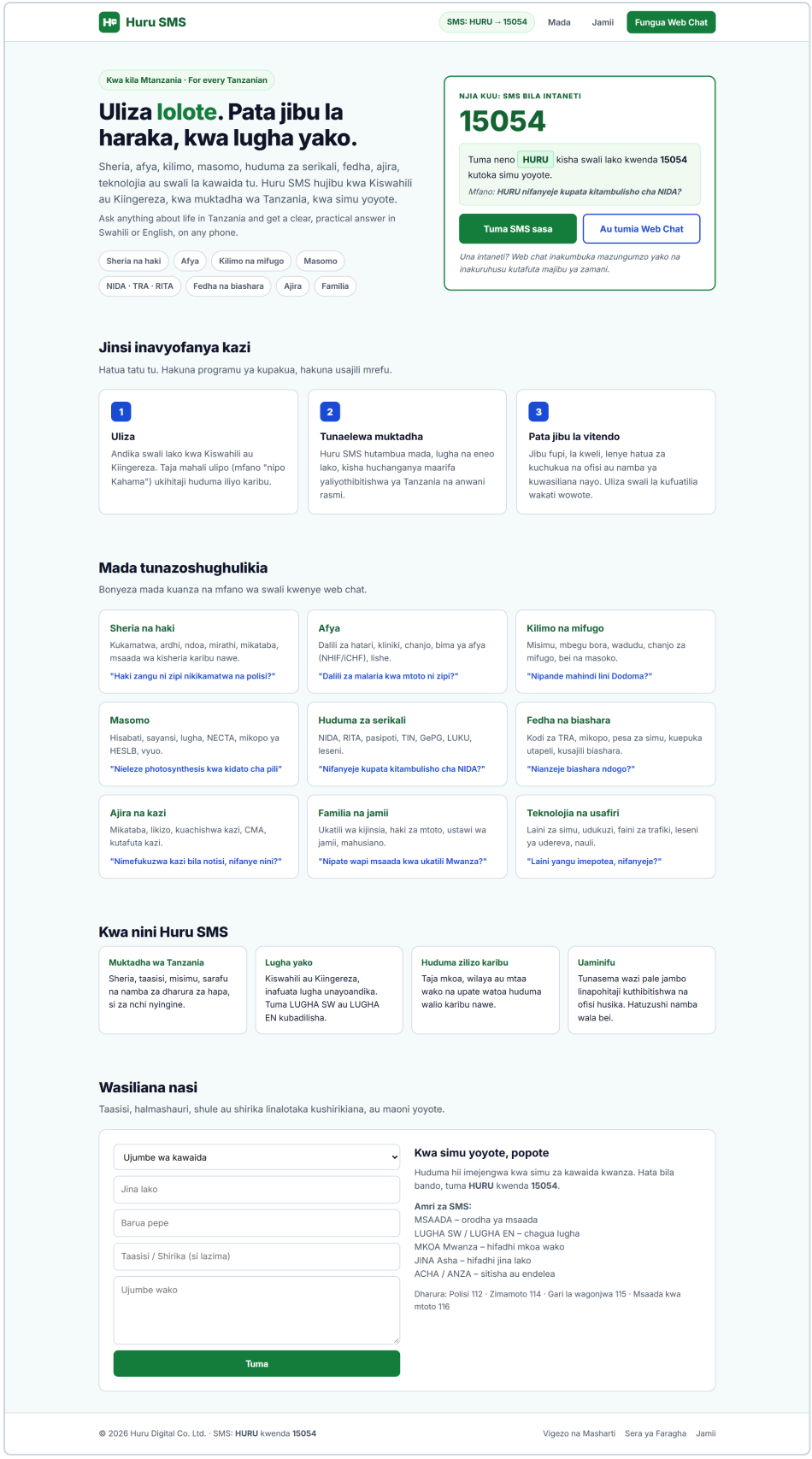


Figure 16. Landing page: the SMS instructions (HURU to 15054), how it works, topics with example questions, and the contact form.

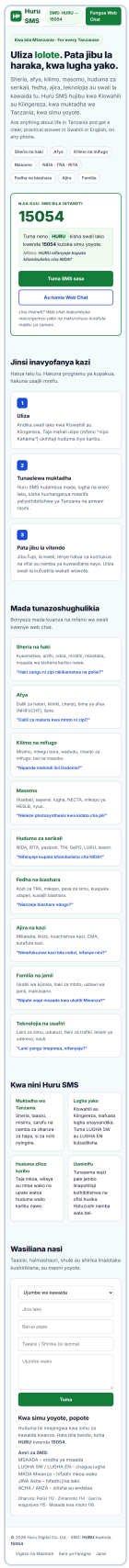


Figure 17. The landing page on a phone.

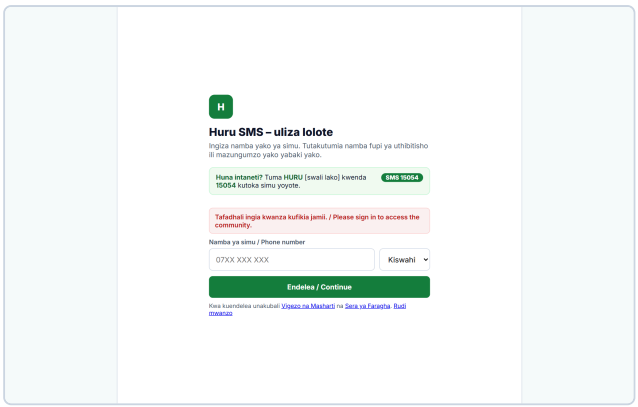


Figure 18. Community board where logged-in citizens share advice in threads.

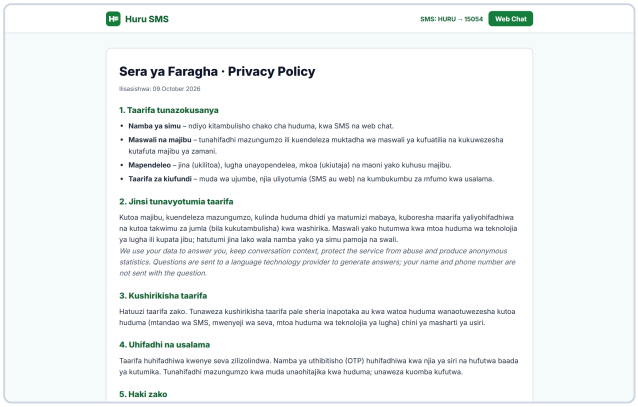


Figure 19. Published privacy policy.

10.4 Administration panel (operator)

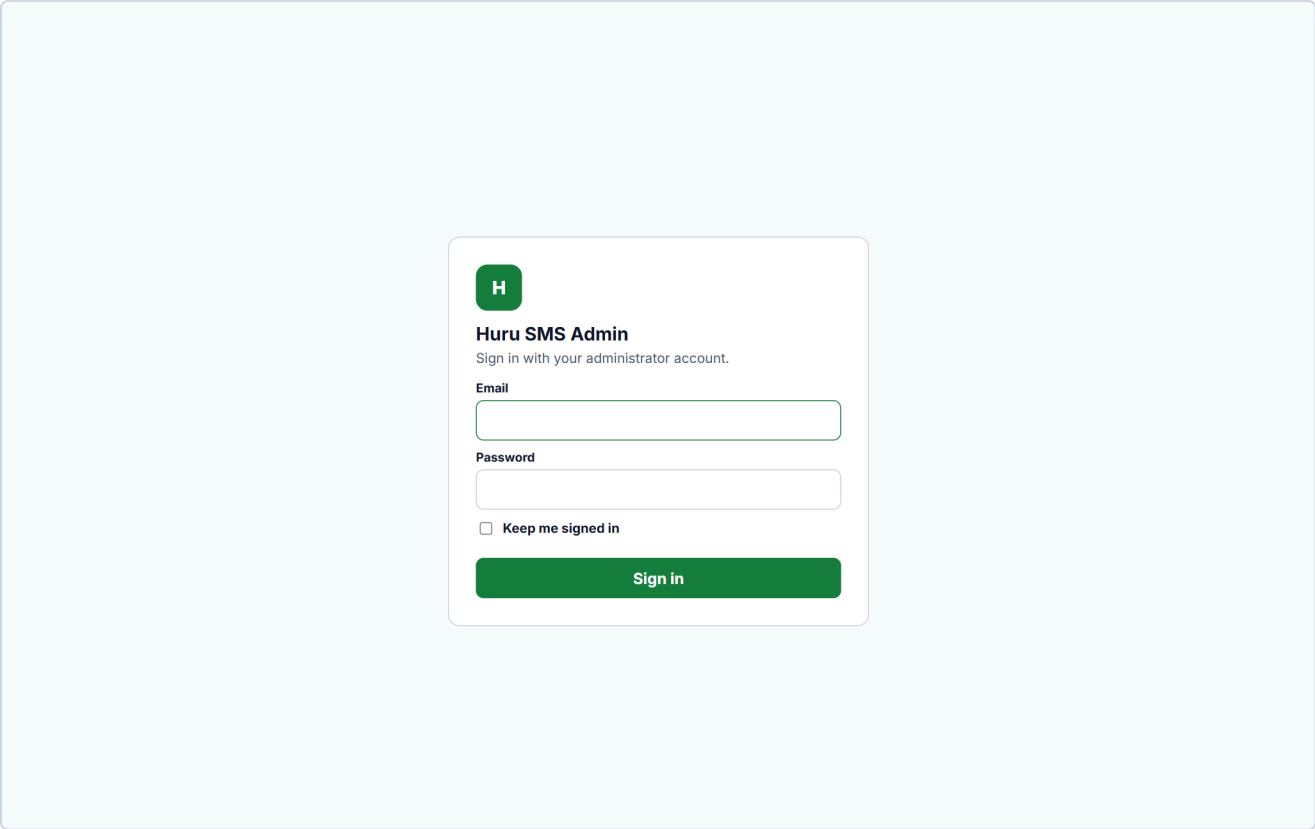


Figure 20. Operator sign-in.

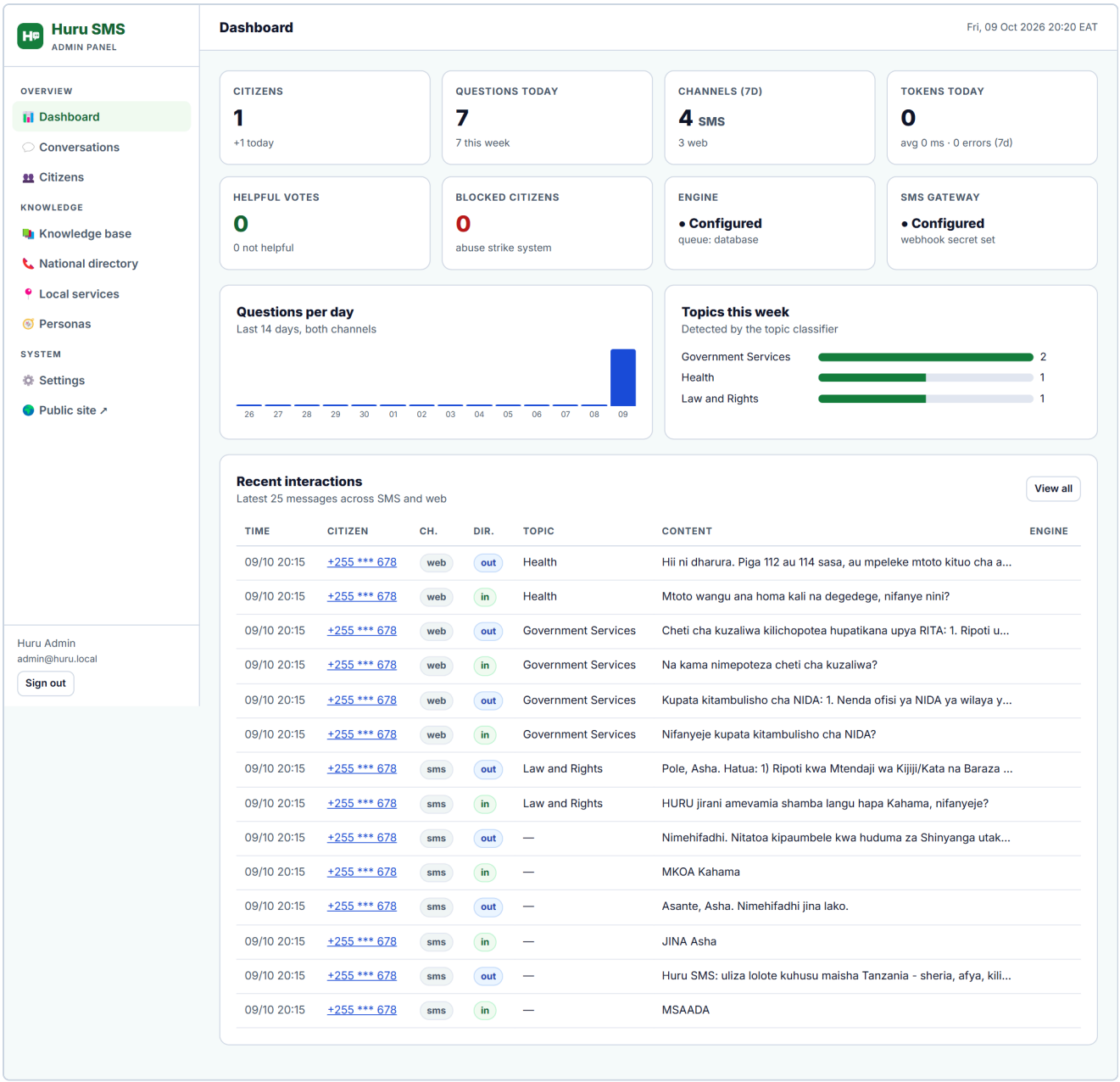


Figure 21. Dashboard: citizens, questions per day, channels, engine and gateway status, topics this week and recent interactions.

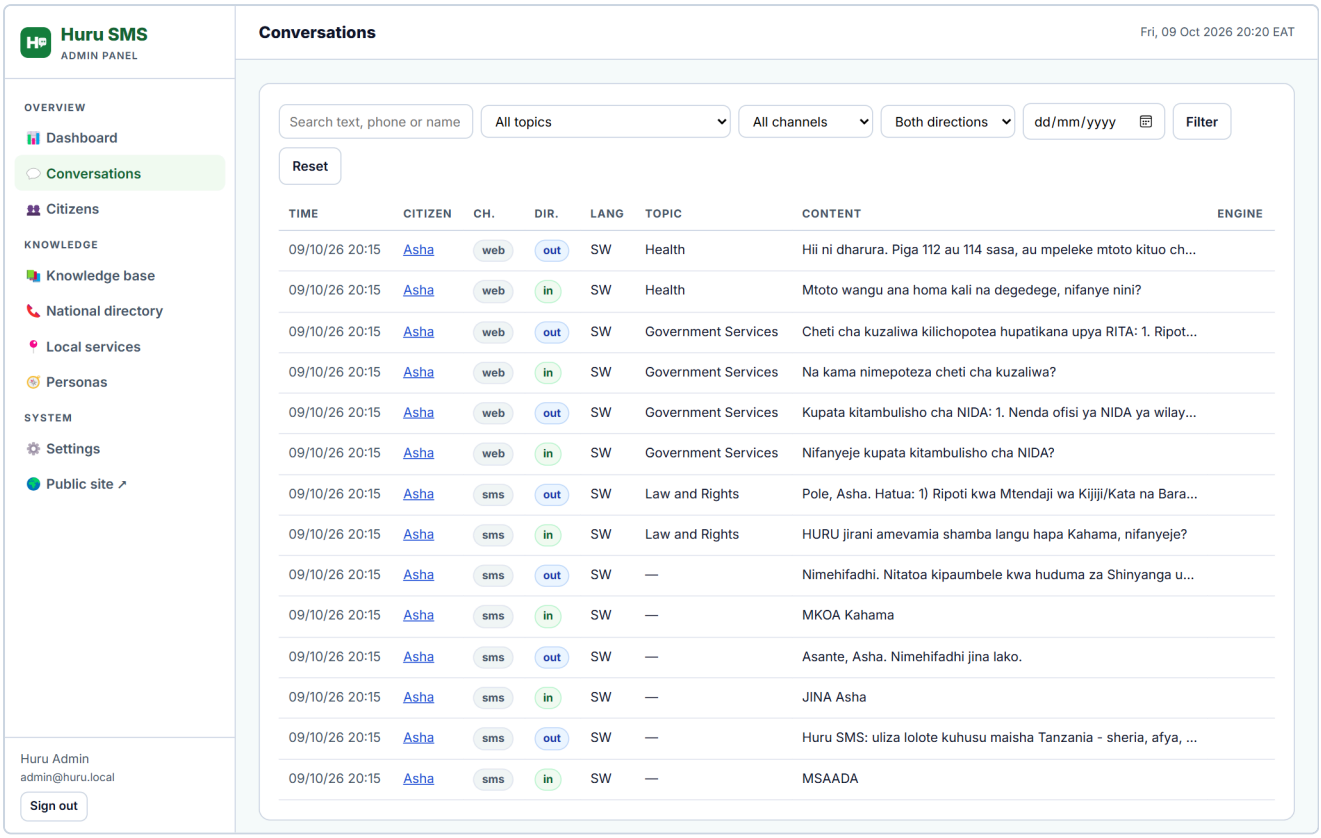


Figure 22. Conversations list with channel and topic filters.

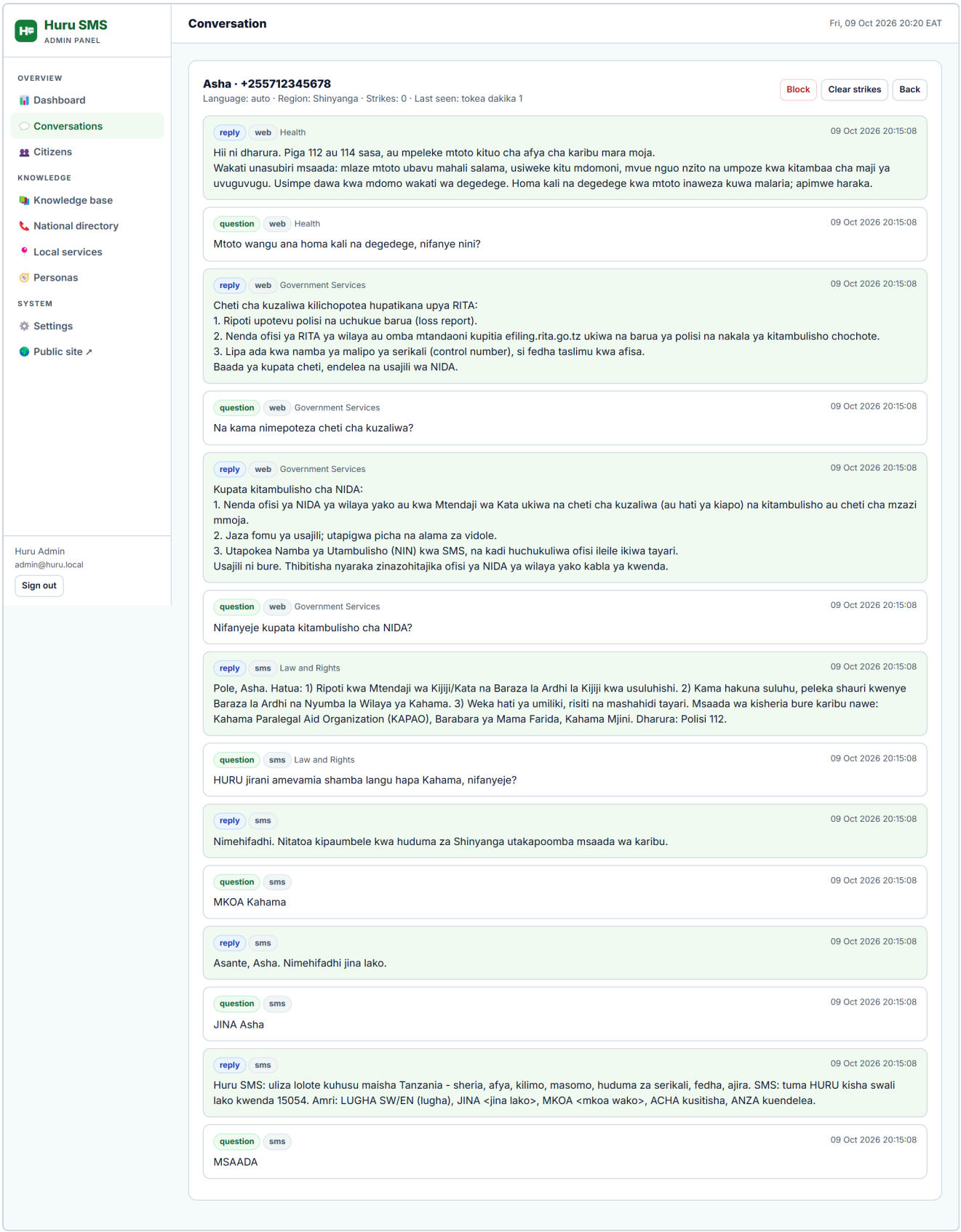


Figure 23. A citizen's conversation with the engine log for each answer (model, tokens, latency, instructions used).

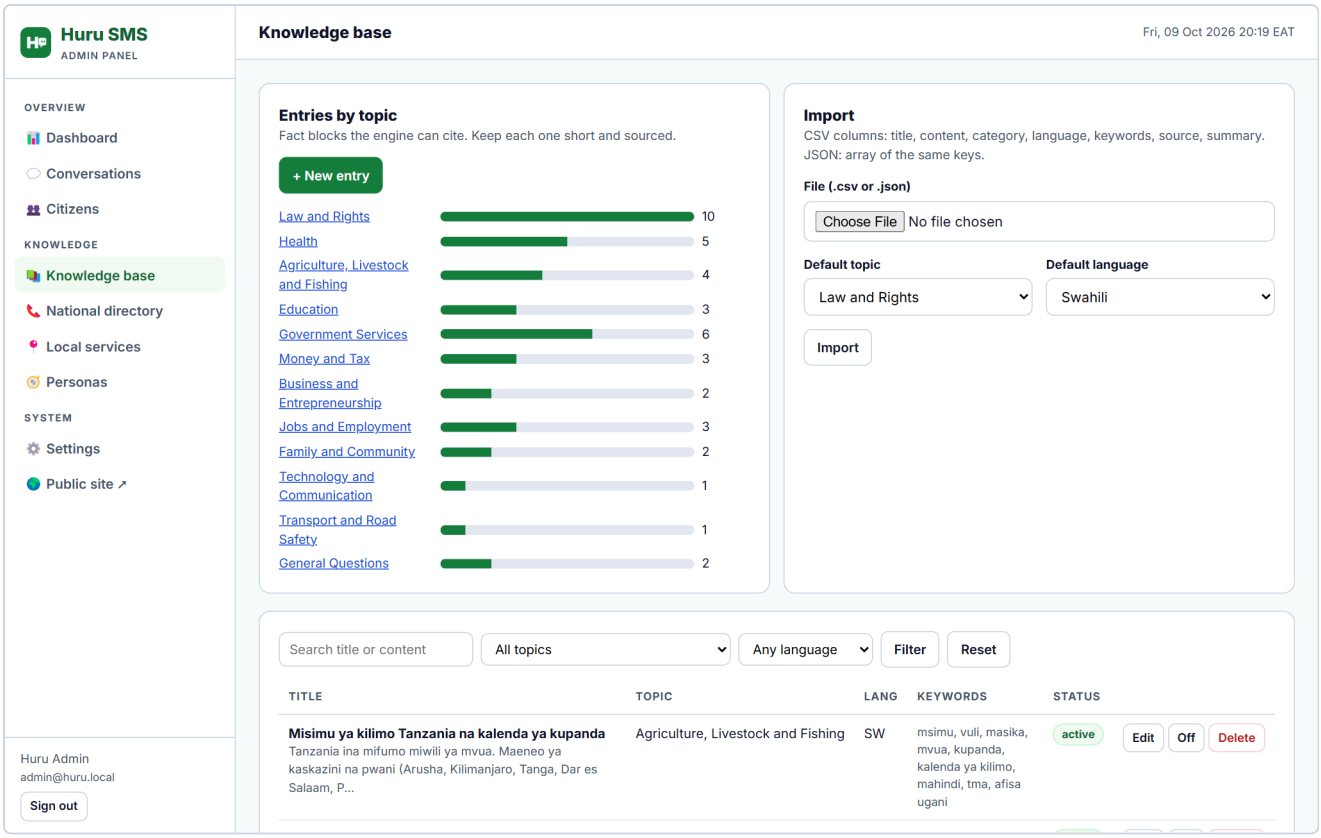


Figure 24. Knowledge base: verified entries by topic and language, with file import.

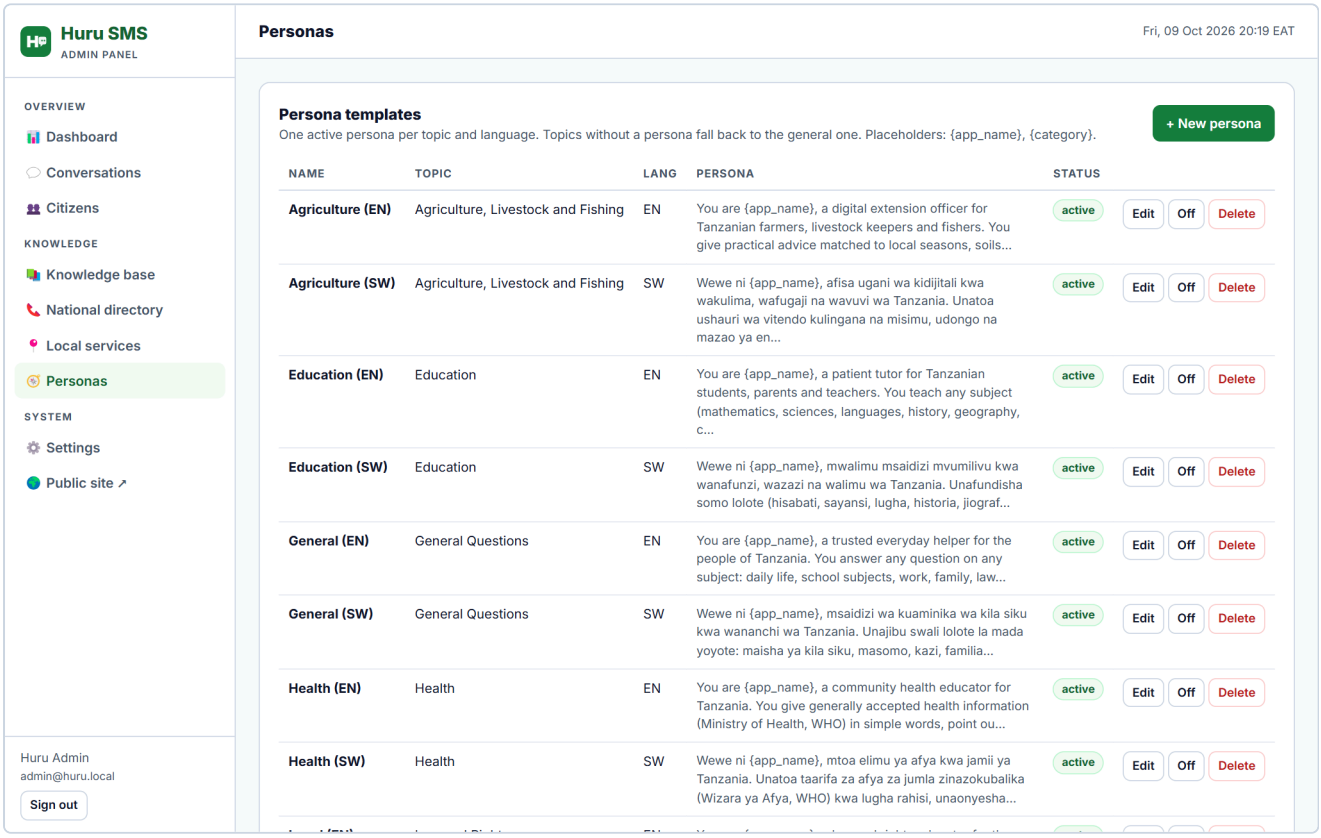


Figure 25. Personas (prompt templates) per topic and language.

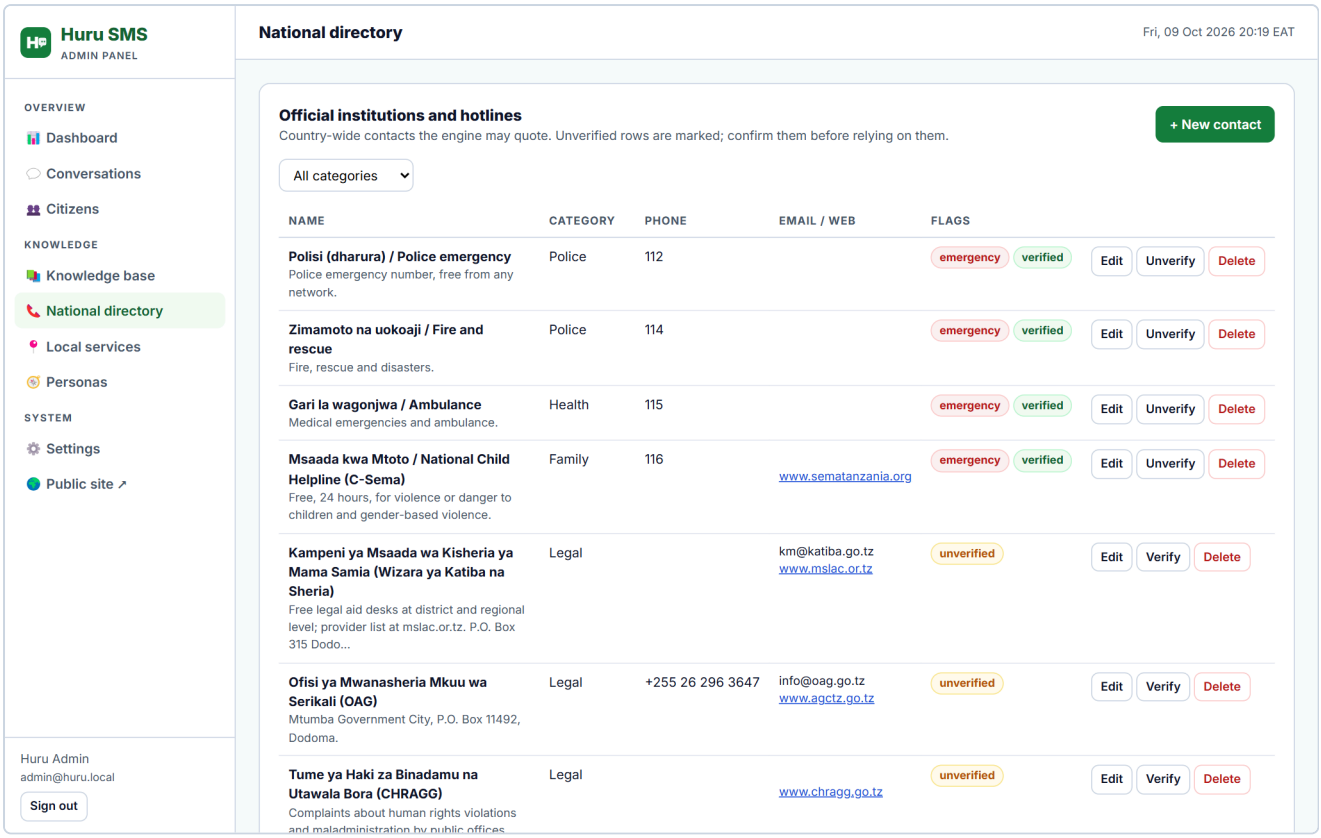


Figure 26. National directory of institutions and hotlines with verification.

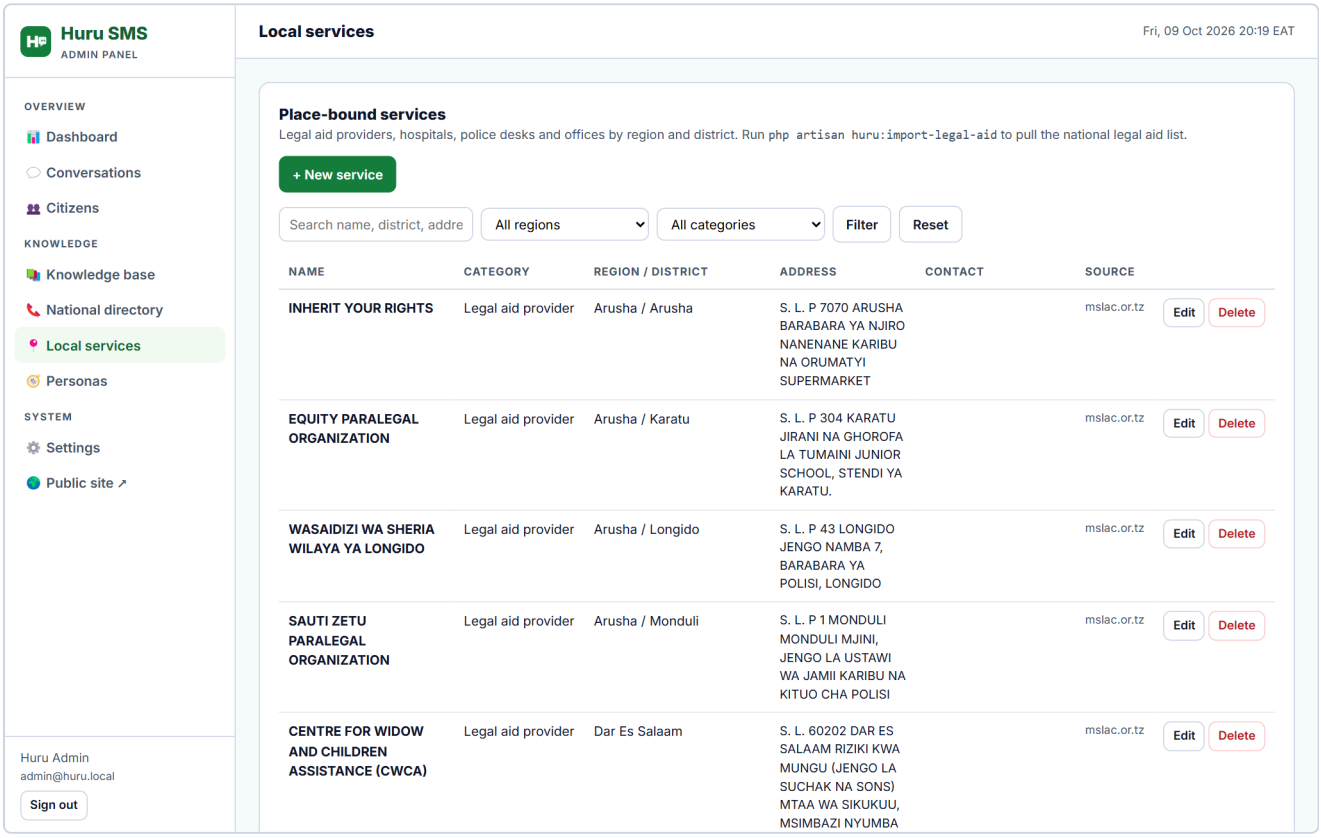


Figure 27. Local services by region and district, including the imported legal aid providers.

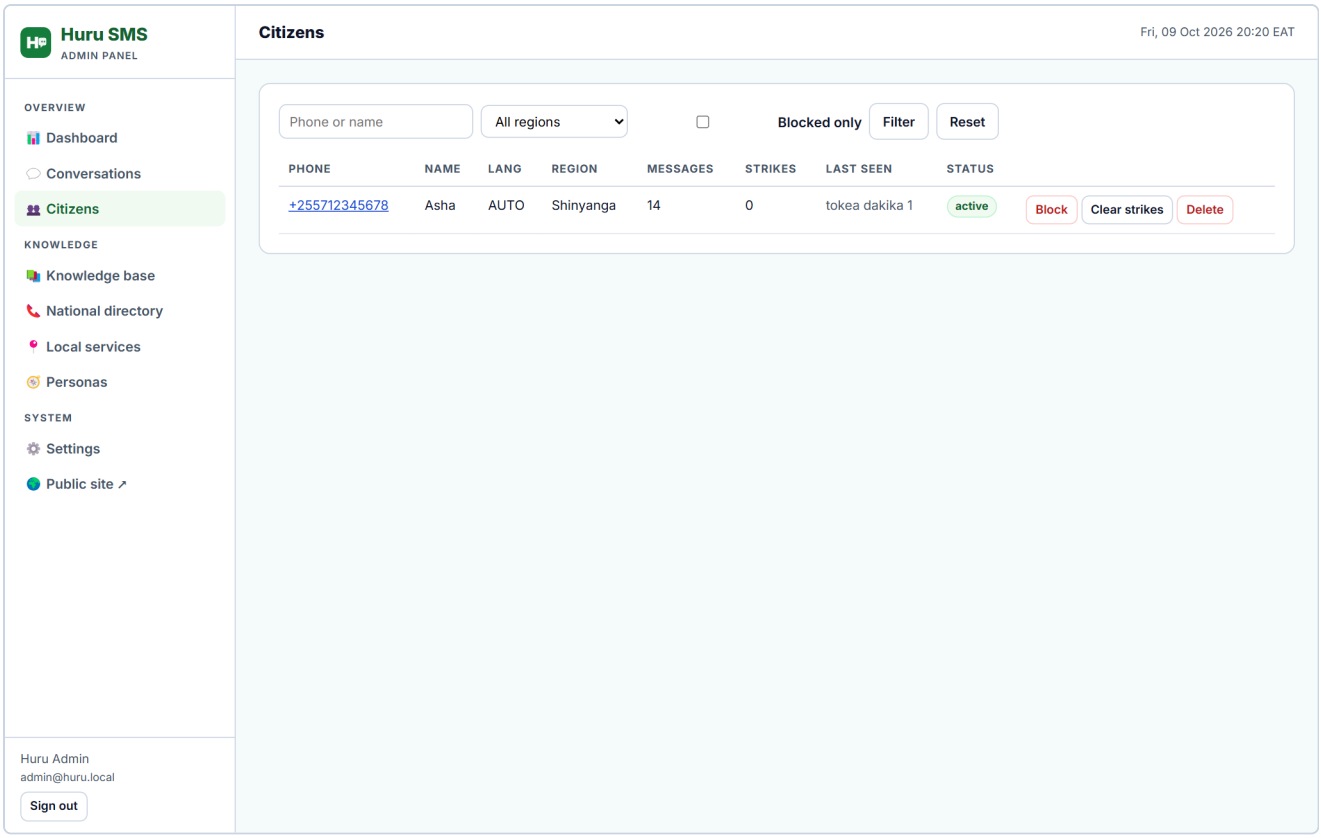


Figure 28. Citizens: activity, strikes, block and unblock.

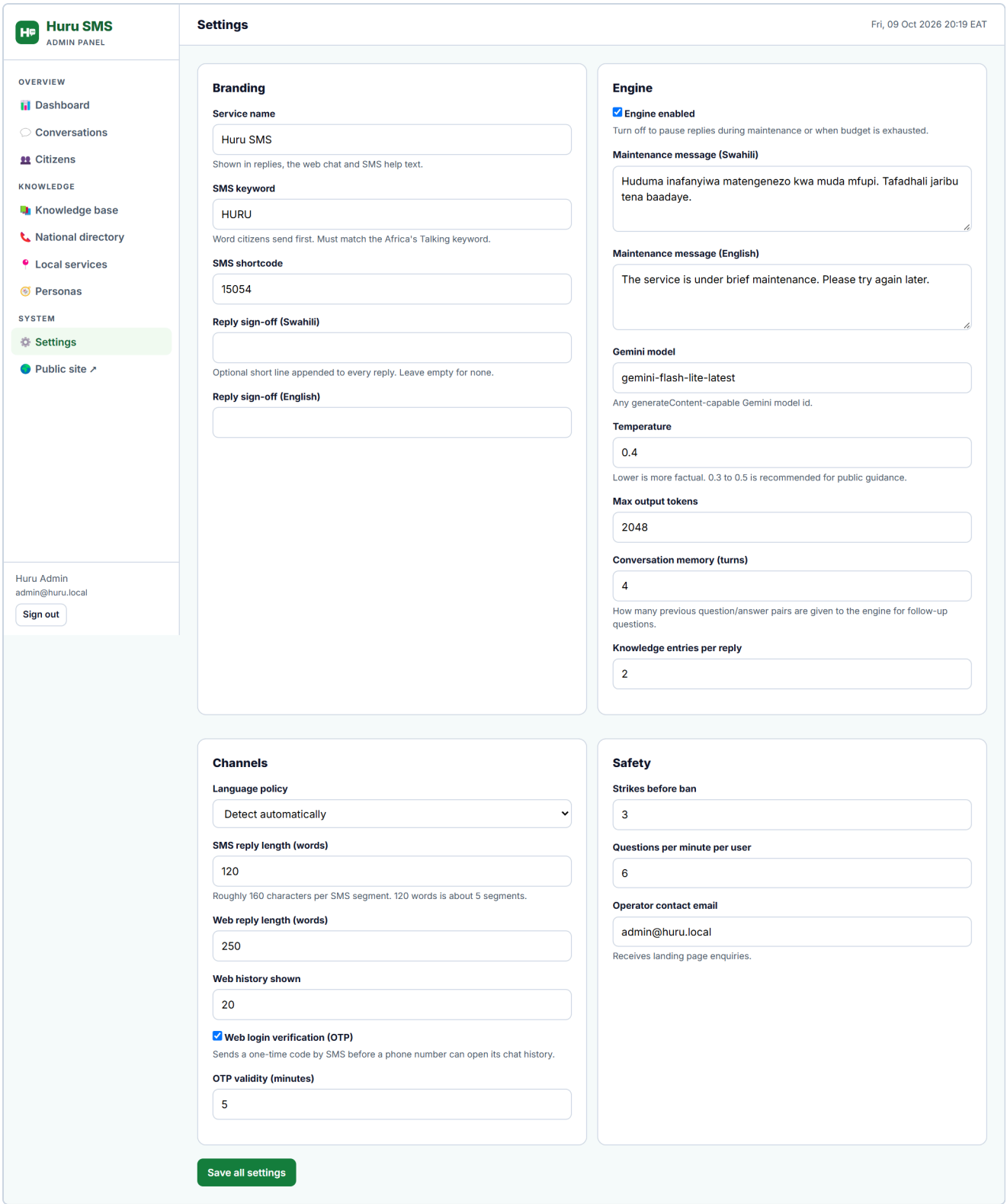


Figure 29. Settings grouped as Branding, Engine, Channels and Safety; only keys in the settings schema can be changed.